Confusingly, though, Procs created with lambda actually differ in some respects from Procs created any other way. The differences are subtle but important enough that people refer to the two kinds of Procs by distinct names: Procs created with lambda are called *lambdas*, while the others are simply called *procs*. (You can use the Proc#lambda? method to check whether the Proc is a lambda.)

One word of warning before you dive into this section: the difference between procs and lambdas is probably the most confusing feature of Ruby, with lots of special cases and arbitrary distinctions. There's no need to go into all the gory details, but you need to know, at least roughly, the important differences.

There are two differences between procs and lambdas. One has to do with the return keyword, and the other concerns the checking of arguments. Let's start with return.

Procs, Lambdas, and return

The first difference between lambdas and procs is that the return keyword means different things. In a lambda, return just returns from the lambda:

blocks/proc_vs_lambda.rb

```
def double(callable_object)
  callable_object.call * 2
end

l = lambda { return 10 }
double(l) # => 20
```

In a proc, return behaves differently. Rather than return from the proc, it returns from the scope where the proc itself was defined:

```
def another_double
  p = Proc.new { return 10 }
  result = p.call
  return result * 2  # unreachable code!
end

another_double # => 10
```

If you're aware of this behavior, you can steer clear of buggy code like this:

```
def double(callable_object)
  callable_object.call * 2
end

p = Proc.new { return 10 }
double(p)     # => LocalJumpError
```